

REPORT: LEZIONE W4D1

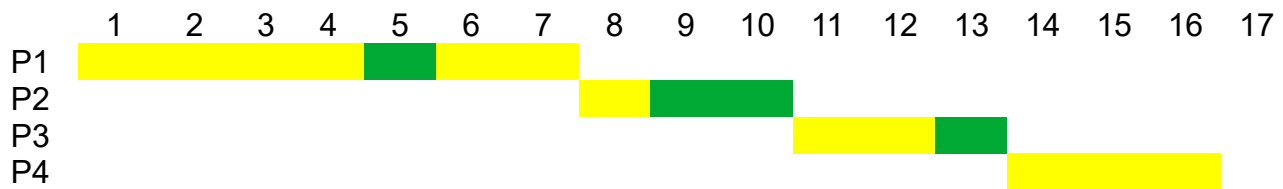
Data la seguente tabella analizzeremo 3 scenari per la valutazione del sistema di pianificazione più efficiente:

- Sistemi mono-tasking
- Sistemi multi-tasking
- Sistemi time-sharing

Processo	Tempo di esecuzione	Tempo di attesa	Tempo di esecuzione dopo attesa
P1	4 secondi	1 secondo	2 secondi
P2	1 secondo	2 secondi	-
P3	2 secondi	1 secondo	-
P4	3 secondi	-	-

LEGENDA
 Cpu
 Attesa

SISTEMI MONO-TASKING



Nei sistemi **mono-tasking**, la CPU viene assegnata a un singolo processo in modo esclusivo. Il limite principale di questa architettura risiede nella gestione dei tempi di attesa: quando un processo si blocca per un'operazione di Input/Output (blocco verde), il processore non può passare a nessun altro compito, rimanendo inattivo e sprecando cicli di calcolo preziosi. I processi successivi rimangono bloccati in coda in stato "Pronto".

SISTEMI MULTI-TASKING

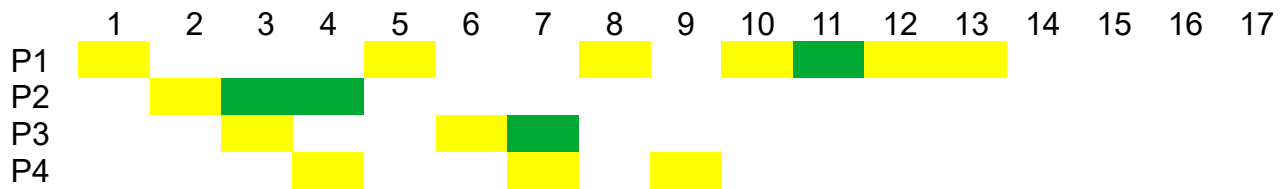


Il sistema **Multi-tasking** introduce una gestione intelligente della CPU basata sull'ottimizzazione dei tempi morti. A differenza del mono-tasking, la CPU non rimane mai inattiva: ogni volta che un processo interrompe il calcolo per effettuare un'operazione di

Input/Output (blocco verde), il sistema operativo gli toglie la CPU e la riassegna subito a un altro processo pronto in coda.

In questo modo, l'attesa I/O di un processo si sovrappone al calcolo di un altro, azzerando gli sprechi temporali.

SISTEMI TIME-SHARING



Il sistema **Time-sharing** introduce un'evoluzione della gestione della CPU basata sulla suddivisione equa del tempo (quantum). A differenza del **multi-tasking** (dove un processo trattiene la CPU finché non va in attesa), la CPU non viene mai monopolizzata da un singolo processo: il sistema operativo assegna a ciascun processo in coda un intervallo di tempo fisso di 1 secondo, facendoli ruotare continuamente.

Se un processo va in attesa per un'operazione di Input/Output (blocco verde), rilascia la CPU in anticipo lasciandola al processo successivo; al contrario, se consuma tutto il suo secondo di calcolo senza andare in attesa, viene temporaneamente sospeso e rimesso in coda per far avanzare gli altri.

In questo modo, le attese I/O si sovrappongono al calcolo altrui e la CPU viene distribuita in modo alternato tra tutti i processi, garantendo risposte rapide e un'equa condivisione delle risorse.

CONSIDERAZIONI FINALI:

Confrontando i tre modelli (**Mono-tasking**, **Multi-tasking** e **Time-sharing**), il sistema **Time-sharing risulta essere il più efficiente in assoluto**. A differenza del multi-tasking (dove la CPU rimane legata a un processo finché questo non decide spontaneamente di fare I/O), nel **Time-sharing** le porzioni di tempo (*quantum*) sono **definite a monte** dal sistema operativo. Questo rende la gestione della CPU prevedibile, controllata e non dipendente dall'efficienza del singolo programma. Inoltre evita che un processo particolarmente lungo o privo di operazioni di I/O "monopolizzi" il processore, garantendo a tutti i processi in coda di avanzare in modo equo e costante.

Per cui il **Time-sharing** combina i vantaggi del multi-tasking (sovrapposizione di calcolo e I/O) con il controllo temporale rigido, assicurando massima fluidità e minimizzando i tempi di attesa percepiti dal sistema.

ESERCIZIO FACOLTATIVO:

- Considerare un insieme di cinque processi P1, P2, P3, P4, P5 con i seguenti tempi di arrivo e di esecuzione (in millisecondi):

Processo	Tempo di arrivo (t_0)	Tempo di esecuzione (T_x)
P1	0	14
P2	30	16
P3	6	40
P4	46	26
P5	22	28

- Descrivere lo scheduling di questi processi con politica **Round Robin** (time slice di **12 millisecondi**).
- Calcolare i tempi di attesa e di turnaround (durata) medi.

t (ms)	CPU	Note / Coda
0 – 12	P1	Arriva P3 (t=6). Coda: P3
12 – 24	P3	Arriva P5 (t=22). Coda: P1, P5
24 – 26	P1	P1 FINISCE (26 ms). Coda: P5, P3
26 – 38	P5	Arriva P2 (t=30). Coda: P3, P2
38 – 50	P3	Arriva P4 (t=46). Coda: P2, P5, P4
50 – 62	P2	Coda: P5, P4, P3
62 – 74	P5	Coda: P4, P3, P2
74 – 86	P4	Coda: P3, P2, P5
86 – 98	P3	Coda: P2, P5, P4
98 – 102	P2	P2 FINISCE (102 ms). Coda: P5, P4, P3
102 – 106	P5	P5 FINISCE (106 ms). Coda: P4, P3
106 – 118	P4	Coda: P3
118 – 122	P3	P3 FINISCE (122 ms). Coda: P4
122 – 124	P4	P4 FINISCE (124 ms). Coda vuota

	t0	tx	tf	turnaround($T_t=T_f-T_0$)	Tempo Attesa($T_a=T_t-T_x$)
P1	0	14	26	26	12
P2	30	16	102	72	56
P3	6	40	122	116	76
P4	46	26	124	78	52
P5	22	28	106	84	56

Tempo di Turnaround Medio: $26 + 72 + 116 + 78 + 84 = 376 : 5 = 75,2$

Tempo di Attesa Medio: $12 + 56 + 76 + 52 + 56 = 252 : 5 = 50,4$

Il **Round Robin** è un algoritmo di scheduling con **preemption** tipico dei sistemi *time-sharing*.

Assegna la CPU ai processi pronti in modo circolare per un intervallo di tempo fisso chiamato **quantum** (*time slice*): se un processo termina o va in I/O prima dello scadere del quantum, rilascia subito la CPU. Se il quantum scade e il processo non è terminato, viene temporaneamente interrotto e posizionato in fondo alla coda dei pronti per cedere il posto al processo successivo.